

Training loops

Module 5 - Lesson 2

Overview

Training adjusts weights to reduce loss. The loop - forward pass, loss, backward pass, update - repeats over batches until the model converges.

Key Concepts

- The forward pass computes predictions; the loss measures the error.
- Backpropagation computes gradients of the loss with respect to every weight.
- Gradient descent updates weights opposite to the gradient scaled by a learning rate.
- Stochastic/mini-batch descent updates on subsets for speed and stability.
- The learning rate controls step size: too large diverges, too small crawls.
- Track training and validation loss per epoch to detect under/overfitting.

Example

```
import torch

optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
loss_fn = torch.nn.MSELoss()

for epoch in range(100):
    y_pred = model(X)
    loss = loss_fn(y_pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    print(epoch, loss.item())
```

Summary

The training loop moves weights down the loss landscape. Backpropagation, a tuned learning rate, and monitoring both training and validation loss produce a model that generalizes.

Practice Checklist

- Train a small network and plot loss per epoch.
- Try two learning rates and compare convergence.
- Watch for divergence (loss exploding) and fix the learning rate.